

Assignment 3: Model-based Reinforcement Learning

Carlo Dubbioso (s4416090) & Maria Mukhovykova (s4379055)

Group Number: 55

1 Introduction

Reinforcement Learning (RL) is a computational framework for automating goal-directed learning through direct interaction with an environment, in which an agent learns to map states to actions so as to maximise a cumulative reward signal [1]. In the previous assignments, we addressed this problem through model-free methods, such as Q-learning. In this assignment, we apply Model-Based Reinforcement Learning (MBRL), in which the agent in addition learns a model of the environment's dynamics and reuses it through planning, updating its value estimates from simulated transitions in between real environment steps. This introduces a central trade-off: planning allows the agent to extract more learning from each real interaction, improving sample efficiency, at the cost of additional computation per step and a dependence on the accuracy of the learned model. We explore two main MBRL algorithms with ϵ -greedy exploration: Dyna and Prioritized Sweeping. Dyna plans from randomly sampled previously-observed transitions, while Prioritized Sweeping focuses its planning effort on the transitions with the largest value changes and propagates these updates backward to predecessor states. For both, we study the effect of the planning budget ("n_planning_updates") and of the degree of environmental stochasticity ("wind_proportion"), comparing each to a model-free Q-learning baseline.

2 The Environment

We evaluate both algorithms on a variant of the Windy Gridworld environment. This environment is a 10×7 grid (70 states) in which the agent can move to an adjacent cell in one of four directions: up, down, left or right. The agent always starts at location (0,3), indicated by 'S', and its goal is to reach location (7,3), indicated by 'G' (we index from 0, as it is in Python). Reaching the goal generates a reward of +100 and terminates the episode, while every other step incurs a reward of -1, which incentivizes the agent to find the shortest path to the goal. What characterizes the environment is the presence of a vertical wind that pushes the agent upward: in columns 3, 4, 5 and 8 the wind displaces the agent by one cell per timestep, and in columns 6 and 7 by two cells. The wind does not blow at every step. The proportion of timesteps on which it blows is controlled by the wind_proportion parameter, which we treat as an experimental variable: setting wind_proportion=0.9 yields a stochastic environment, in which the agent feels the wind on most steps but experiences windstill conditions on the remaining 10%, making the transition function non-deterministic. Setting wind_proportion=1.0 makes the wind blow at every step, recovering a fully deterministic environment in which the agent must still learn to compensate for the constant upward drift.

3 Dyna

3.1 Methodology

Dyna is a model-based algorithm that expands the standard Q-learning with planning: it learns a model of the environment, and subsequently use this model to make additional one-step planning updates to our value function. We keep the model as two tables, the transition counts $n(s, a, s')$ and the accumulated reward sums $R_{\text{sum}}(s, a, s')$, both initialized to zero. After observing a transition (s, a, r, s') , we update these counts:

$$n(s, a, s') \leftarrow n(s, a, s') + 1$$

$$R_{\text{sum}}(s, a, s') \leftarrow R_{\text{sum}}(s, a, s') + r$$

from which we obtain the estimated transition distribution and the estimated reward function:

$$\hat{p}(s' | s, a) = \frac{n(s, a, s')}{\sum_{s''} n(s, a, s'')} \\ \hat{r}(s, a, s') = \frac{R_{\text{sum}}(s, a, s')}{n(s, a, s')}$$

Each environment step is used for the update of the Q-table

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

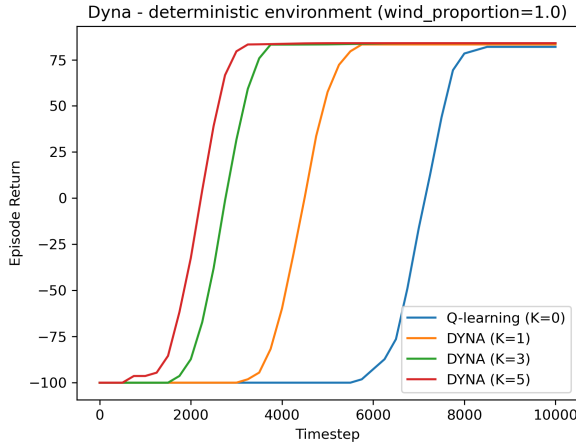
where α is the learning rate, γ the discount factor, and the term within brackets the temporal difference error.

We then perform K planning updates (n_planning_updates), and for each: we sample at random a state-action pair that we previously observed, we simulate the environment $s', r \sim \hat{p}(s', r | s, a)$ from the learned model, and apply the same Q-learning update using the simulated transition. Actions are selected through a ϵ -greedy policy.

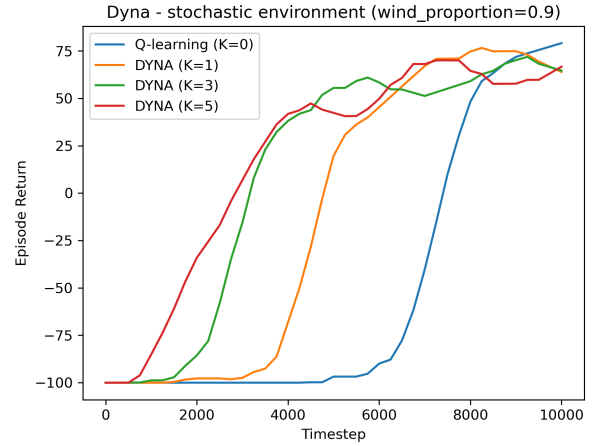
We evaluate the agent every 250 timesteps by running 30 greedy episodes (maximum length 100) and recording the mean return. We then average all learning curves over 20 independent repetitions, each with a freshly initialized environment and agent, and then smoothed. For this experiment we fixed the following hyperparameters to: $\epsilon = 0.1$, $\alpha = 0.2$ and $\gamma = 1.0$; and compared planning budgets $K \in \{0, 1, 3, 5\}$ on both stochastic (wind_proportion=0.9) and deterministic (wind_proportion=1.0) environments.

3.2 Results

In the deterministic environment (Figure 1a), the planning budget produces a clear ordering: all configurations start at a return of -100 and at different speed they all converge close to a return of 80. Dyna with the planning update $K=5$ is the fastest to reach the highest reward approximately in 3500 timesteps, followed by $K=3$ and finally $K=1$, while the model-free baseline $K=0$ begins to improve only around timestep 6000. This shows that planning is a vital



(a) Dyna in the deterministic environment



(b) Dyna in the stochastic environment

Figure 1: Dyna

component to reduce the number of steps actually needed to learn the optimal policy without undermining the final performance.

In the stochastic environment (Figure 1b), the plot result more noisy. The learning order is approximately similar to the deterministic environment: higher planning budget accelerates the initial learning and return, with $K=5$ followed by $K=3$ and $K=1$; but the asymptotic behavior is opposite. The highest planning budget flattens earliest and at the lowest final return, whereas the lowest non-zero budget $K=1$ continues improving and achieves the best final return, close to +75. The model-free baseline $K=0$, despite being the slowest to learn, ultimately reaches a final return slightly below the best planning configuration.

3.3 Interpretation

The reason planning helps is sample efficiency. Every real transition costs an environment step, but planning updates are cheap, because they reuse the learned model to replay experience without interacting with the environment. This is why a larger planning budget speeds up learning. In between each real step the agent performs more updates from its model, spreading newly observed reward information through the value function more quickly, so it needs fewer real steps to find a good policy. We see this most clearly in the deterministic environment. Here the wind always blows, so the learned transition model $\hat{p}(s'|s, a)$ quickly becomes exact, and every planning update replays a perfectly reliable transition. The only effect of a larger budget is then faster propagation, which is exactly what Figure 1 shows: the curves keep the same final plateau but rise sooner the higher the budget.

The stochastic environment shows the main weakness of planning from a learned model. Now the wind only blows 90% of the time, so the same state-action pair can lead to different next states, and the model $\hat{p}(s'|s, a)$ has to estimate a whole distribution from a limited number of counts. This means the model is only an approximation, especially early on when the agent has visited each pair just a few times. Every planning update bootstraps from this imperfect

model, so the more we plan, the more updates we make against the same model error before a real transition comes along to correct it. This is exactly why $K=5$ learns fastest at the start but ends up with the worst final policy: heavy planning propagates information quickly, but it also amplifies the model's bias. A smaller budget like $K=1$ relies more on real experience and is less distorted by the model, which is why it keeps improving and reaches the highest final return. The model variability is also why the stochastic curves are much noisier than the deterministic ones.

4 Prioritized Sweeping

4.1 Methodology

Prioritized Sweeping is a reinforcement learning algorithm that can be considered as an improved version of Dyna. While Dyna selects state-action pairs randomly, Prioritized Sweeping makes planning updates more efficiently by focusing on the state-action pairs' temporal difference (TD) error and choosing larger one. This means that it puts more effort on computing the Q-value estimates of each pair and prioritizing the estimate which most likely would have significant change.

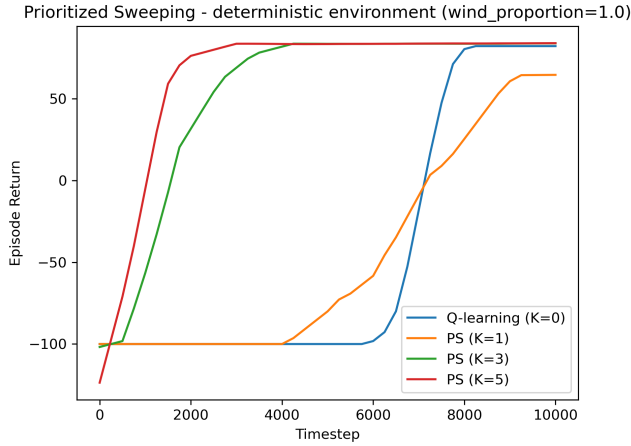
Similarly to Dyna algorithm, the agent has the learning model of the environment with the transition counts $n(s, a, s')$ and cumulative reward sums $R_{sum}(s, a, s')$. From these we can define transition function and reward function:

$$\hat{p}(s' | s, a) = \frac{n(s, a, s')}{\sum_{s''} n(s, a, s'')}$$

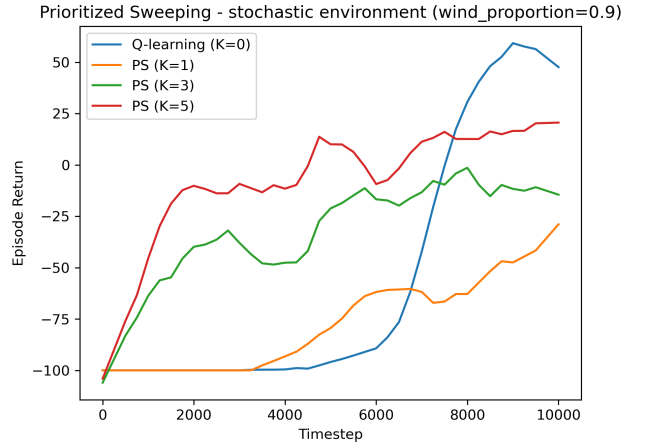
$$\hat{r}(s, a, s') = \frac{R_{sum}(s, a, s')}{n(s, a, s')}$$

However, the valuable addition is Priority Queue (PQ); after each real environment step (s, a, r, s') , the TD error is calculated and defined as priority p :

$$p = \left| r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right|$$



(a) Prioritized Sweeping in the deterministic environment



(b) Prioritized Sweeping in the stochastic environment

Figure 2: Prioritized Sweeping

- If p is large that means the Q -value is not very accurate and it needs an update and it is added to the queue with high priority.
- If p is small, then Q -value is correct and does not need prioritizing.

Afterwards the planning loop starts where the highest priority pair (s, a) is taken from the queue, the transition function is applied followed by estimated reward calculation, and finally the Q -learning updated is performed:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\hat{r}(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Additionally, this algorithm looks backwards, meaning for every predecessor $(\bar{s}, \bar{a})$ with $n(\bar{s}, \bar{a}, s) > 0$ it computes new priority:

$$\bar{p} = \left| \hat{r}(\bar{s}, \bar{a}, s) + \gamma \max_{a'} Q(s, a') - Q(\bar{s}, \bar{a}) \right|$$

If the new priority $(\bar{p})$ is large, then the predecessor is added to the queue and the loop continues. Exactly this backward propagation makes Prioritized Sweeping especially strong, because after state's value is updated, the information propagates to all states and makes future planning stay where it is needed.

4.2 Results

Two learning curves for Prioritized Sweeping with different planning budgets K are shown above. The algorithm was run with $\epsilon = 0.1$, $\alpha = 0.2$, $\gamma = 1.0$ in both stochastic and deterministic environments. As well, for both scenarios the Q -learning baseline $K = 0$ serves as the lower point on performance.

The first plot (Figure 2a) shows deterministic environment with $\text{wind_proportion} = 1.0$, we can clearly see that planning improves learning. When $K = 5$, the algorithm finds the optimal path by ~ 1500 timestep, with $K = 3$ it needs ~ 2500 , and plain Q -learning ~ 8000 . In conclusion, more planning $\approx$ faster learning, because the

model is accurate and every update is trustworthy. However, the exception can be observed with $K = 1$, because one planning step per update step is not enough to create great difference.

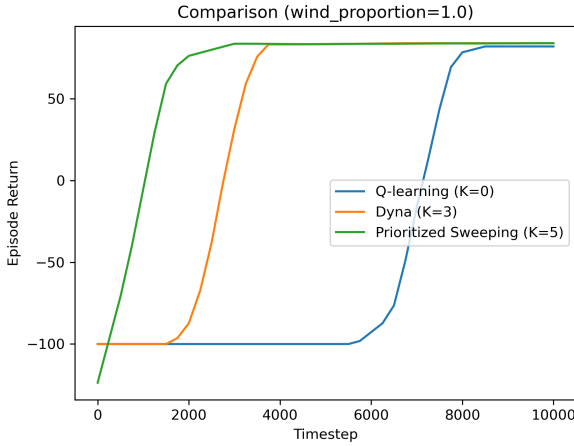
In the second plot (Figure 2b) the stochastic environment with $\text{wind_proportion} = 0.9$ is presented. Here we can see that with $K = 5$ and $K = 3$, the algorithm learns faster in the beginning, but Q -learning catches up and has the best result. However, Prioritized Sweeping with $K = 1$ performs the worst, this happens due to the environment's randomness, the model is not perfectly accurate, and the planning can point the agent in the wrong direction. Q -learning does not build the model at all, meaning that it is not affected by the randomness, as it learns from experience and ends up with better performance.

4.3 Interpretation

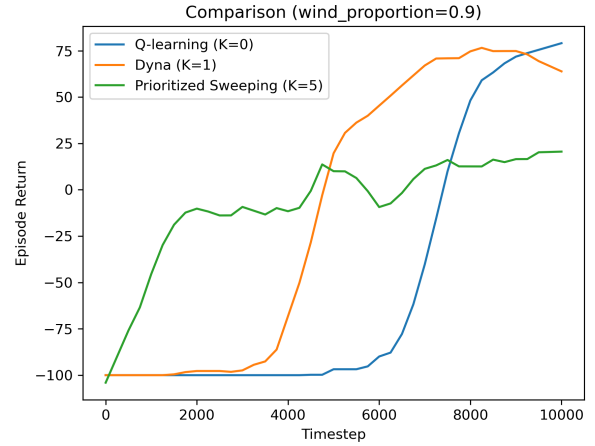
From the results we can observe that Prioritized Sweeping has more advantage compared to Q -learning and Dyna algorithm. By focusing on state-action pairs with higher potential for Q -value and this way improving planning, the algorithm takes more useful signal from each planning step.

The most important mechanism in prioritized Sweeping is backward propagation, especially it is valuable in Windy Environment. Due to reward being scattered, Q -values near the goal are the first to become informative. Algorithm propagates this information backwards, so the states couple of steps away from goal getting rapid updating and this makes Prioritized Sweeping more efficient than Dyna's random sampling.

However, in stochastic environment, the priority queue can be filled with misleading updates, which decreases advantage of this algorithm compared to Dyna. Because the model is uncertain, this leads to the TD errors being large not because Q -value is far from optimal, but because the environment's behavior is random every step. So, the Prioritized Sweeping spends planning budget in updates that are not very useful, and this makes Q -learning to perform



(a) Comparison of algorithms in the deterministic environment



(b) Comparison of algorithms in the stochastic environment

Figure 3: Comparison

better in the long run, as it relies in actual experience and can't be misled by model inaccuracy.

5 Comparison

5.1 Results

Two comparison plots (Figure 3) present the best-performing variant of Prioritized Sweeping and Dyna algorithms compared to the Q-learning baseline in both environments: stochastic and deterministic.

Deterministic environment (wind_proportion = 1.0): Prioritized Sweeping with $K = 5$ converges fastest and gains almost optimal return by ~ 2000 timestep. Afterwards goes Dyna algorithm with $K = 3$, it converges around 3800 timestep. Meanwhile Q-learning is the slowest one and it converges only at ~ 8000 timestep. Although, all three algorithms reach approximately the same result, the difference is in how fast they get there.

Stochastic environment (wind_proportion = 0.9): In this environment the whole picture changes. Prioritized Sweeping with $K = 5$ starts learning fastest, but after ~ 2000 timestep it settles around ~ 35 by 10000 timestep. Here Dyna is with $K = 1$ and it starts slower, but by timestep 8000 it overtakes Prioritized Sweeping and gets to ~ 75 return. Q-learning is the slowest algorithm to start learning; however, it still reaches ~ 80 by the end and finishes roughly the same as Dyna. Finalizing, in the stochastic environment Q-learning and Dyna algorithms perform approximately equally; moreover, they both have higher performance over Prioritized Sweeping.

5.2 Runtime

From runtime table (Table 1) we can see that Prioritized Sweeping is by far the slowest algorithm in both environments. This happens because after each planning update, there must be computed priorities of all predecessor states in the model, and this is quite expensive. Dyna is the fastest one in stochastic environment as it uses only $K = 1$ planning step and it is computationally cheap.

Algorithm	Stoch. (s)	Det. (s)
Q-learning ($n=0$)	2.94	1.85
Dyna ($n=1/3$)	2.55	2.10
PS ($n=5/5$)	4.38	3.83

Table 1: Average runtime per repetition.

5.3 Interpretation

In the deterministic environment, both Model Based Reinforcement Learning algorithms perform better than Q-learning, especially Prioritized Sweeping is the best. This shows that the main advantage of such algorithms is that the reusing experience through planning and the agents needs less steps to converge.

However, in the stochastic environment, the model-based algorithms show significant disadvantage. Prioritized Sweeping performs the worst even though it is the fastest to start learning. Dyna algorithm performs better, but likely because of the only one random planning update step that does not rely on inaccurate model. In this environment Q-learning becomes competitive with Dyna showing that with unpredictable environment learning from the experience is the safest option.

6 Reflection

6.1 Model-based vs. Model-free RL

Sample efficiency is the main advantage of model-based RL. The agent can learn more from each real experience when it is building the model of the environment and planning with it. This was visible in deterministic environment where both Prioritized Sweeping and Dyna converged faster than Q-learning using the same amount of real timesteps.

However, when the environment is stochastic the model cannot be perfectly accurate and can plan based on wrong estimates which can mislead the agent. The model-based RL depends on the accuracy

of the learned model, and this is the main weakness. We observed that Q-learning outperformed Prioritized Sweeping in stochastic environment and was not weaker than Dyna algorithm. As well, Prioritized Sweeping was much slower than Q-learning as building and storing a model is computationally expensive.

6.2 Dyna vs. Prioritized Sweeping

As mentioned previously, Prioritized Sweeping outperformed Dyna in the deterministic environment, as it converges faster due to its priority-driven planning. While in stochastic environment Dyna showed better results because its random planning works better with inaccurate model.

Which idea is better? Prioritized Sweeping is theoretically cleaner because it focuses on the most important updates by computing their priorities and then propagates information backwards. Dyna as well is good algorithm but simpler and more robust. In the end, the choice should be made based on the environment.

Moreover, these model-based algorithms can be combined. For instance, Dyna's random sampling could be used as a substitution when the priority queue in Prioritized Sweeping is empty. Also, it could be used when the uncertainty increases, and random sampling can slowly reduce dependency on the model. This combined

approach can be beneficial for Prioritized Sweeping in the deterministic environment while keeping Dyna's robustness in stochastic settings.

6.3 Exploration and Q-value Initialization

Initially all $Q(s, a)$ values equal to 0 and default reward for every step is -1 . So, if the agent has not yet visited the state, its value is 0, which is higher than the true value of visited states. This encourages exploration, because the agent wants to go through unvisited states and naturally explore new parts of the grid.

In case if we set default reward to 0 instead of -1 , all Q-values would remain at 0 no matter if they were visited or not. This removes the exploration part of the algorithm, as both visited and unvisited states look the same for the agent, and it has no desire to explore new states. Without such exploration initiative, the learning would slow down, as the agent would likely revisit the same states repeatedly which would take longer to find the goal. The -1 default reward is not only a time penalty but also important aspect to stimulate exploration.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.